

Data types in details

Strings

Strings are arrays containing bytes that represent Unicode characters. In Python, there is no data type named character. Characters are strings of length one.

For more basic information about what is a string in python please refer to [Strings in Python](#)

Note : `str()` is a keyword in python, so it is good not to use it as a variable name.

Creating a String

- Strings are created by enclosing characters inside single quotes, double quotes, or triple quotes.
- Triple Quotes declaration is used when we need to write multi-line strings.

```
# Defining strings In Python
# Declaration using single quote
new_string = 'Hello World'
print(new_string)
new_string = "Hello World"
print(new_string)
new_string = """Hello World,
                Do you know what is a string in python?"""
print(new_string)
```

Accessing Characters in Python

- We can access individual characters using indexing and the range of characters using slicing.
- Index starts from 0.
- Trying to access a character out of index range will raise an **IndexError**.
- **Index** must be an integer.
- Python allows negative indexing for its sequences.
 - The last item has an index of -1, the second last item has an index of -2, etc.

• String	H	e	l	l	o		W	o	r	l	d
Positive Index	0	1	2	3	4	5	6	7	8	9	10
Negative Index	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
# Accessing string characters in Python
```

```
string = 'Hello World'
```

```
print('string = ', string)
```

```
# First character
```

```
print('string[0] = ', string[0])
```

```
# Last character
```

```
print('string[-1] = ', string[-1])
```

```
# Slicing 3rd to 6th character
```

```
print('string[2:6] = ', string[1:5])
```

```
# Slicing 5th to 2nd last character
```

```
print('string[4:-2] = ', string[5:-2])
```

String Slicing

To access a range of characters in the string, slicing is used. The **Slicing operator** is used to slice a String i.e. **colon**.

```
# Accessing string characters in Python
```

```
string = 'Hello World'
```

```
print('string = ', string)
```

```
# Slicing 3rd to 6th character
```

```
print('string[2:6] = ', string[1:5])
```

```
# Slicing 5th to 2nd last character
```

```
print('string[4:-2] = ', string[5:-2])
```

String Immutability

Strings in Python are immutable. This means that they cannot be changed after they are created. If we need to manipulate strings then we can use methods

like **concatenation**, **slicing**, or **formatting** to create new strings based on the original.

```
s = "welcome"
```

```
# Trying to change the first character raises an error
```

```
# s[0] = 'I' # Uncommenting this line will cause a TypeError
```

```
# Instead, create a new string
```

```
s = "W" + s[1:]
```

```
print(s)
```

Output

```
Welcome
```

Deleting/Updating from a String

- We cannot update or delete characters of strings after declaration as strings in python are immutable.
- Deletion of an entire string is possible with the help of the **del** keyword.

```
string = 'Hello World'
print('string = ', string)

# Updating character of string
# Error will be displayed
string[0] = 'A'
print('String after updation of character is', string)

# Deleting character of a string
# Error will be displayed
del string[2]
print('String after deletion of character is', string)

# Updating Whole String
string = 'Scalar'
print('String after updation is', string)
```

```
# Deleting the whole string  
# Error will be displayed  
del string  
print('String after updation is', string)
```

Concatenation of Two or More Strings

We can concatenate two strings with the help of the '+' operator. The '*' operator can also be used to make a string repeat a certain number of times.

```
# using + operator (Concatenation)  
string1 = 'Hello'  
string2 = 'World'  
print('String after concatenation is:', string1+string2)  
  
# using * operator (Repetition)  
print('String after 4 times of repetition is:', string1*4)
```

Iterating Through a string

There are two methods to iterate through the string with the help of for loop.

```
string = 'Hello World'  
print('Method 1')  
for i in string:  
    print(i)  
print('Method 2')  
for i in range(len(string)):  
    print(string[i])
```

Python String Length

We can find string length using two methods

Method - 1: Maintaining count variable

```
string = 'Hello World'  
count = 0
```

```
for i in string:  
    count += 1  
print('Length of string is', count)
```

Method - 2 : Using built in [len\(\) function](#)

```
string = 'Hello World'  
print('Length of string is', len(string))
```

Python String Formatting

String formatting can be performed using two methods :

Method - 1 : using [format\(\)](#) method

- It specifies the values and inserts them inside the string's placeholder.
- Placeholder is defined using curly brackets : {}.
- **Syntax :** string.format(value1, value2,). Values are inserted in the string.

```
string = 'Hello World'  
print('Length of {} is {}'.format(string, len(string)))
```

Method - 2: formatting string using F-string

```
string = 'Hello World'  
print(f'Length of {string} is {len(string)}')
```

upper() and lower(): [upper\(\)](#) method converts all characters to uppercase. [lower\(\)](#) method converts all characters to lowercase.

s = "Hello World"

`print(s.upper())` # output: HELLO WORLD

`print(s.lower())` # output: hello world

Output

HELLO WORLD

hello world

strip() and replace(): strip() removes leading and trailing whitespace from the string and replace(old, new) replaces all occurrences of a specified substring with another.

```
s = "  Gfg  "
```

```
# Removes spaces from both ends
```

```
print(s.strip())
```

```
s = "Python is fun"
```

```
# Replaces 'fun' with 'awesome'
```

```
print(s.replace("fun", "awesome"))
```

Output

```
Gfg
```

```
Python is awesome
```

Using **in** and **not in** for String Membership Testing

The **in** keyword checks if a particular substring is present in a string.

```
s = "GoodforGood"
```

```
print("Good" in s)
```

```
print("GfG" in s)
```

Output

```
True
```

```
False
```

List in Python

In Python, a **list** is a built-in dynamic sized array (automatically grows and shrinks). We can store all types of items (including another list) in a list. A list may contain mixed type of items, this is possible because a list mainly stores references at contiguous locations and actual items maybe stored at different locations.

- List can contain duplicate items.
- List in Python are Mutable. Hence, we can modify, replace or delete the items.
- List are ordered. It maintain the order of elements based on how they are added.

- Accessing items in List can be done directly using their position (index), starting from 0.

The data stored in the List is ordered. That is, it will be stored in the way we input it. Therefore, Lists are ordered sequences. List in Python also offers various simple and shorthand ways like List comprehensions.

Characteristics of a Python List

The various characteristics of a list are:

- **Ordered:** Lists maintain the order in which the data is inserted.
- **Mutable:** In list element(s) are changeable. It means that we can modify the items stored within the list.
- **Heterogenous:** Lists can store elements of various data types.
- **Dynamic:** List can expand or shrink automatically to accommodate the items accordingly.
- **Duplicate Elements:** Lists allow us to store duplicate data.

Creating Lists in Python

A list is created by placing the items/ elements separated by a comma (,) between the square brackets ([]). Let's see how we can create Lists in Python in different ways.

```
# Creation of a List in Python
```

```
# Creating an empty List  
empty_List = []
```

```
# Creating a List of Integers  
integer_List = [26, 12, 97, 8]
```

```
# Creating a List of floating point numbers  
float_List = [5.8, 12.0, 9.7, 10.8]
```

```
# Creating a List of Strings  
string_List = ["Interviewbit", "Preparation", "India"]
```

```
# Creating a List containing items of different data types  
List = [1, "Interviewbit", 9.5, 'D']
```

```
# Creating a List containing duplicate items
duplicate_List = [1, 2, 1, 1, 3,3, 5, 8, 8]
```

We can also create a list storing several other lists as its items. It is known as a nested list. The nested list is created by placing the various lists separated by comma (,) within the square brackets ([]). Let us see below how nested lists are created in Python.

```
# Creating a nested List
nested_List = [[6, 2, 8], [1, "Interviewbit", 3.5], "preparation"]
```

Using list() Constructor

We can also create a list by passing an **iterable** (like a **string**, **tuple**, or another **list**) to **list()** function.

```
# From a tuple
```

```
a = list((1, 2, 3, 'apple', 4.5))
```

```
print(a)
```

Output

```
[1, 2, 3, 'apple', 4.5]
```

Creating List with Repeated Elements

We can create a list with repeated elements using the multiplication operator.

```
# Create a list [2, 2, 2, 2, 2]
```

```
a = [2] * 5
```

```
# Create a list [0, 0, 0, 0, 0, 0, 0]
```

```
b = [0] * 7
```



```
print(a)
```

```
print(b)
```

Output

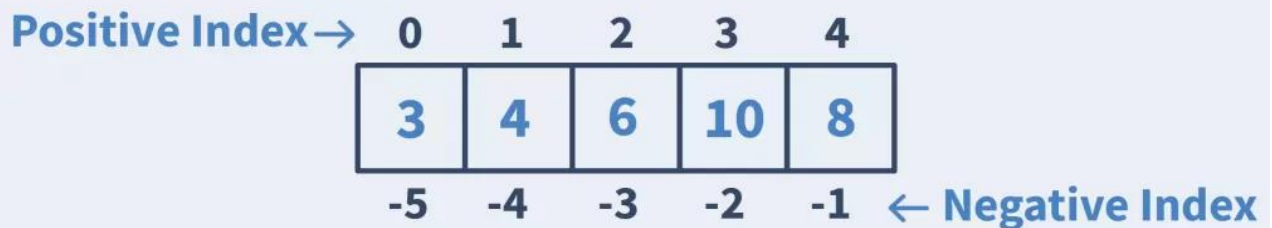
```
[2, 2, 2, 2, 2]  
[0, 0, 0, 0, 0, 0, 0]
```

Accessing Values/Elements in List in Python

Elements stored in the lists are associated with a unique integer number known as an index. The first element is indexed as 0, and the second is 1, and so on. So, a list containing six elements will have an index from 0 to 5. For accessing the elements in the List, the index is mentioned within the index operator ([]) preceded by the list's name.

Another way we can access elements/values from the list in python is using a negative index. The negative index starts at -1 for the last element, -2 for the last second element, and so on.

My_List = [3, 4, 6, 10, 8]



```
My_List = [3, 4, 6, 10, 8]
```

```
# Accessing values stored in lists using positive index  
print("Values accessed using positive Index.")
```

```
print(My_List[2])
print(My_List[4])

# Accessing values stored in lists using negative index
print("Values accessed using negative Index.")
print(My_List[-1])
print(My_List[-5])
```

OUTPUT:

```
Values accessed using positive Index.
6
8
Values accessed using negative Index.
8
3
```

We also have nested Lists in python. For nested Lists, we first access the index at which the inner list is stored/indexed in the outer list using the index operator. Then we access the element from the inner list using the index operator again. It is known as nested indexing. It is like we access elements in a 2-dimensional array. Unlike in this, we have many elements in the inner list, but arrays only have a single value at that index.

```
# Nested List
nested_List = [[6, 2, 8], [1, 3.5, "Interviewbit"], "preparation"]
print(nested_List[0][1])
print(nested_List[1][2])
```

OUTPUT:

```
2
Interviewbit
```

Updating List in Python

Lists are mutable, meaning that their existing values stored at the indexes can be changed. To change or update the list elements, [the assignment operator \(=\)](#) is used.

Let's see the code below that shows how it's actually done.

```
My_List = [3, 4, 6, 10, 8, 5]
print(My_List)
# Changing value to index 3.
My_List[3] = 7
print(My_List)
# Changing value to index -1, i.e the last value.
My_List[-1] = 15
print(My_List)
```

OUTPUT:

```
[3, 4, 6, 10, 8, 5]
[3, 4, 6, 7, 8, 5]
[3, 4, 6, 7, 8, 15]
```

Add/Change list Elements in Python

Lists are mutable, unlike strings, so we can add elements to the lists. We use the `append()` method to add a single element at the end of the list and the `extend()` method to add multiple elements at the end of the list. We also have an `insert()` method to insert an element at the specified index position. The `insert()` method takes two arguments, the first is the index at which the element is to be inserted, and the second is the value/ element to be inserted.

```
vowels = ['a', 'e']
print(vowels)
# Appending an element in list
vowels.append('i')
print(vowels)
# Extending an element in list
vowels.extend(['o', 'u'])
print(vowels)
# Inserting an element at a specified index
Integers = [1, 5, 6, 9]
Integers.insert(1,7)
print(Integers)
```

OUTPUT:

```
['a', 'e']  
['a', 'e', 'i']  
['a', 'e', 'i', 'o', 'u']  
[1, 7, 5, 6, 9]
```

Delete/Remove Lists Elements in Python

We can delete one or multiple elements from the list in python using the del keyword. We can even delete the whole list using this keyword. If we delete a list and then try to access it, the interpreter will generate an error.

Now let's see an example code below, how the del keyword is used.

```
# Deleting/Removing items from the List.  
my_List = [1, 2, 3, 4, 5, 6, 7, 8]  
print(my_List)  
# Deleting/Removing a single item from the list  
del my_List[3]  
print(my_List)  
# Deleting the entire List  
del my_List  
# The below line will generate an error as the List doesn't exist now.  
print(my_List)
```

OUTPUT:

```
[1, 2, 3, 4, 5, 6, 7, 8]  
[1, 2, 3, 5, 6, 7, 8]  
  
NameError: name 'my_List' is not defined  
  print(my_List)  
Line 10 in <module> (Solution.py)
```

We have three more methods remove(), pop(), and clear() for this purpose.

- The remove() method takes the item as an argument that must be deleted or removed. If the item isn't present within the list, a ValueError is raised.
- The pop() method takes the index as an argument and deletes and returns the deleted item. If the index isn't given, pop() deletes and returns the list's last element.
- The clear() method in python makes the list empty.

```
# Deleting/Removing items from List using various methods
my_List = [1, 2, 3, 4, 5, 6, 7, 8, 9]
print(my_List)
# Deleting/Removing using remove() method
my_List.remove(4)
print(my_List)
# Deleting/Removing using pop() method
print(my_List.pop(1))
print(my_List)
# pop() method with no arguments
print(my_List.pop())
print(my_List)
# clear() method
my_List.clear()
print(my_List)
```

OUTPUT:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
[1, 2, 3, 5, 6, 7, 8, 9]
2
[1, 3, 5, 6, 7, 8, 9]
9
[1, 3, 5, 6, 7, 8]
[]
```

List Operations in Python

Let's see the various operations we can perform on the list in Python.

- **Concatenation:** One list may be concatenated with itself or another list using '+' operator.

```
List1 = [1, 2, 3, 4]
List2 = [5, 6, 7, 8]
concat_List = List1 + List2
print(concat_List)
```

OUTPUT:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

- **Repeating the elements of the list:** We can use the ‘*’ operator to repeat the list elements as many times as we want.

```
my_List = [1, 2, 3, 4]
print(my_List*2)
```

OUTPUT:

```
[1, 2, 3, 4, 1, 2, 3, 4]
```

- **Membership Check of an element in a list:** We can check whether a specific element is a part/ member of a list or not.

```
my_List = [1, 2, 3, 4, 5, 6]
print(3 in my_List)
print(10 in my_List)
print(10 not in my_List)
```

OUTPUT:

```
True
False
True
```

- **Iterating through the list:** We can iterate through each item within the list by employing a for a loop.

```
my_List = [1, 2, 3, 4, 5]
for n in my_List:
    print(n)
```

OUTPUT:

```
1
2
3
4
5
```

- **Finding the length of List in python:** `len()` method is used to get the length of the list.

```
my_List = [1, 2, 3, 4, 5]
print(len(my_List))
```

OUTPUT:

```
5
```

Slicing a List in Python

In Python, we can access the elements stored in a range of indexes using the slicing operator (:). The index put on the left side of the slicing operator is inclusive, and that mentioned on the right side is excluded.

Slice itself means part of something. So when we want to access elements stored at some part or range in a list we use slicing.

```
my_List = ['i', 'n', 't', 'v', 'i', 'e', 'w', 'b', 'i', 't']
# Accessing elements from index beginning to 5th index
print(my_List[:6])
# Accessing elements from index 3 to 6
print(my_List[3:7])
# Accessing elements from index 5 to end
print(my_List[5:])
# Accessing elements from beginning to end
print(my_List[:])
# Accessing elements from beginning to 4th index
print(my_List[:-6])
```

OUTPUT:

```
['i', 'n', 't', 'v', 'i', 'e']
['v', 'i', 'e', 'w']
['e', 'w', 'b', 'i', 't']
['i', 'n', 't', 'v', 'i', 'e', 'w', 'b', 'i', 't']
['i', 'n', 't', 'v']
```

We can also use slicing to change, remove/delete multiple elements from the list. Let's see some code examples and understand them.

```
List = [1, 2, 3, 4, 5, 6, 7, 8]
print(List)
```

```
# Changing elements from index 2nd to 5th
List[2:5] = [1, 1, 1]
print(List)
# Deletion multiple items from the List
del List[2:6]
print(List)
```

OUTPUT:

```
[1, 2, 3, 4, 5, 6, 7, 8]
[1, 2, 1, 1, 1, 6, 7, 8]
[1, 2, 7, 8]
```

List Methods in Python

In python, the method is like a function that performs some task but can only be called on objects. Methods are related to objects and may change the object's state. Let's see some python list methods.

S. No.	Method	Description of method
1	append()	Add an item to the ending of the list.
2	extend()	Add items of one list to another list.
3	insert()	Adds an item at the desired index position.
4	remove()	Remove the primary item from the list with the required value.
5	pop()	Remove the item from the required index within the list, and return it.
6	clear()	Remove all items from the list.
7	index()	Returns 0-based index of the first matched item. Raises ValueError if an item isn't found within the list.
8	count()	Returns the frequency of an element appearing in the list.
9	sort()	Sorts the objects of the list. It uses a compare function if passed as an argument.

S. No.	Method	Description of method
10	reverse()	Reverse the order of the list items in place.
11	copy()	Return a shallow copy of the list i.e. two lists share the identical elements via reference.

Built-in List Methods/Functions in Python

Functions are a group of instructions that typically operate on sequences or objects in python. Python provides plenty of built-in functions for lists. Here is a list of some built-in functions associated with lists in python.

S. No.	Function	Description
1	len(my_List)	Calculates and returns the length or size of the list
2	max(my_List)	Returns the item with maximum value within the list
3	min(my_List)	Returns the item with minimum value within the list
4	list(iterable)	Return the list containing the elements of the iterable.
5	filter(function,sequence)	Test each element of the list as true or not on the function passed as an argument thereto and returns a filtered iterator.
6	map(function, iterator)	Returns a list after applying the function passed as an argument to every element of the iterable.
7	lambda()	An anonymous function that may have any number of arguments but one expression, that calculates and returns the result.
8	all()	Returns True if all the items/elements present within the list are true or if the list is empty.

S. No.	Function	Description
9	any()	Returns True if any of the item(s)/at least one among the items present within the list is/are true. And returns False if the list is empty.
10	sum()	Returns the sum of all the elements present within the list.
11	reduce(func, sequence)	It applies a function passed as an argument to all the elements of the sequence and returns a result based on the function.

Iterating Over a Python List

The list is a type of ordered container that can store heterogeneous and homogenous data. Every element stored in the list has a definite position and a definite count. There are various ways in which lists can be traversed/iterated. As we advance, we will be going through various practices of iterating through a list

- **By using for loop:** We make use of for loop to traverse from index 0 to end.

```
# Creating List
my_List = ['i', 'n', 't', 'e', 'r', 'n']
# Traversing using for loop.
for character in my_List:
    print(character, end=' ')
```

OUTPUT:

```
i n t e r n
```

- **By using for loop and range():** We use range() to traverse over the specified indexes or traverse list traditionally. However it is advised to iterate over a list using the previous method, instead of using index in this method.

```
# Initializing a List
my_List = ['i', 'n', 't', 'e', 'r', 'n']
size = len(my_List)
```

```
# Traversing using for loop.  
for i in range(size):  
    print(my_List[i], end = '')
```

OUTPUT:

```
i n t e r n
```

- **By using while loop:**

```
# Initializing a List  
my_List = ['i', 'n', 't', 'e', 'r', 'n']  
size = len(my_List)  
i = 0  
# Traversing using a while loop.  
while i < size:  
    print(my_List[i], end = ' ')  
    i += 1
```

OUTPUT:

```
i n t e r n
```

- **List comprehension:** Using List comprehension is one of the best possible ways to iterate over a list. List comprehension makes the program look more elegant and easily understandable.

```
my_List = ['i', 'n', 't', 'e', 'r', 'n']  
# iterating using List Comprehension  
[print(x, end = ' ') for x in my_List]
```

OUTPUT:

```
i n t e r n
```

- **Using the enumerate():** `enumerate()` in python allows us to loop through elements as well as the index within the list. This is helpful when we want to traverse only certain index elements in the list based on certain condition(s).

```
my_List = ['i', 'n', 't', 'e', 'r', 'n']  
# iterating using enumerate
```

```
for i, item in enumerate(my_List):  
    if i%2 == 0:  
        print(i, item)
```

OUTPUT:

```
0 i  
2 t  
4 r
```

Advanced Python List Processing

Sorting a list

To sort a list in ascending or descending order, we use the [sort\(\)](#) function with the following syntax:

For ascending order:

```
list.sort()
```

For descending order:

```
list.sort(reverse=True)
```

Example:

```
# sorting a list using sort() function  
  
my_list = [5, 2, 90, 24, 10]  
  
# sorting in ascending order permanently  
  
# changes the order of the list  
  
my_list.sort()  
  
print(my_list)  
  
# sorting in descending order permanently  
  
# changes the order of the list
```

```
my_list.sort(reverse=True)
```

```
print(my_list)
```

Output:

```
[2, 5, 10, 24, 90]
```

```
[90, 24, 10, 5, 2]
```

To temporarily change the order of the list use the *sorted()* function with the syntax:

```
list.sorted()
```

```
# temporary sorting using sorted() method
```

```
my_list = [5, 2, 90, 24, 10]
```

```
# ascending order
```

```
print(sorted(my_list))
```

```
# descending order
```

```
my_list_2 = sorted(my_list)
```

```
print(my_list)
```

Output:

```
[2, 5, 10, 24, 90]
```

```
[5, 2, 90, 24, 10]
```

Reversing a list

To reverse the order of a list we use the *reverse()* function. Its syntax is:

```
list.reverse()
```

Example:

```
# reverse a list using reverse()

my_list = [5, 2, 90, 24, 10]

# reverse

my_list.reverse()

print(my_list)
```

Output:

```
[10, 24, 90, 2, 5]
```

Or we could apply list comprehension to reverse a list:

```
list = list[::-1]
```

Example:

```
# reverse using list comprehension

my_list = [5, 2, 90, 24, 10]

# reverse

print(my_list[::-1])
```

Output:

```
[10, 24, 90, 2, 5]
```

Removing duplicates

As dictionaries in python do not contain duplicate keys. We use the *dict.fromkeys()* function to convert our list into a dictionary with list elements as keys. And then we convert the dictionary back to list.

This is a powerful trick to automatically remove all the duplicates. Its syntax is:

```
My_list=['a', 'b', 'c', 'b', 'a']  
Mylist = list(dict.fromkeys(My_List))
```

Example:

```
# removing duplicates from a list using dictionaries  
  
my_list_1 = [5, 2, 90, 24, 10, 2, 90, 34]  
  
my_list_2 = ['a', 'a', 'a', 'b', 'c', 'd', 'd', 'e']  
  
# removing duplicates from list 1  
  
my_list_1 = list(dict.fromkeys(my_list_1))  
  
print(my_list_1)  
  
# removing duplicates from list 2  
  
my_list_2 = list(dict.fromkeys(my_list_2))  
  
print(my_list_2)
```

Output:

```
[5, 2, 90, 24, 10, 34]  
['a', 'b', 'c', 'd', 'e']
```

Filtering a list

Python lists can be filtered with the help of *filter()* function or with the help of list comprehension. Below is the syntax:

```
My_list = list(filter(filter_function , iterable_item))
```

The Filter function returns an iterator object which we must convert back into the list.

Example:

```
# filtering with the help of filter() function

# creating a filter function filter all the values less than 20

# filter function

def my_filter(n):

    if n > 20:

        return n

# driver code

if __name__ == "__main__":

    my_list = [5, 2, 90, 24, 10, 2, 95, 36]

    my_filtered_list = list(filter(my_filter, my_list))

    print(my_filtered_list)
```

Output:

```
[90, 24, 95, 36]
```

We can also filter using list comprehension. It is a much easier and elegant way to filter lists, below is the syntax:

```
My_list = [item for item in my_list if (condition)]
```


Example:

```
# filtering with the help of list comprehension

my_list = [5, 2, 90, 24, 10, 2, 95, 36]

# an elegant way to sort the list

my_list = [item for item in my_list if item > 20]

print(my_list)
```

Output:

```
[90, 24, 95, 36]
```

Modifying list

To modify the values in the list with the help of an external function, we make use of the map() function. Map() function returns a map object (iterator) of the results after applying the given function to each element of a given iterable(list, tuple etc.). Below is the syntax:

```
My_list = list(map(function,iterable))
```

Example:

```
# using map() function to modify the text

def squaring(n):

    return n**2

# driver code
```

```
if __name__ == "__main__":  
  
    my_list = [5, 2, 90, 24, 10, 2, 95, 36]  
  
    my_squared_list = list(map(squaring, my_list))  
  
    print(my_squared_list)
```

Output:

[25, 4, 8100, 576, 100, 4, 9025, 1296]

A much cleaner approach is using list comprehension.

Example

```
# the same result can be obtained by a much pythonic approach  
  
# i.e., by using list comprehension  
  
my_list = [5, 2, 90, 24, 10, 2, 95, 36]  
  
print([i**2 for i in my_list])
```

Output:

[25, 4, 8100, 576, 100, 4, 9025, 1296]

Combining lists

We can even combine lists with the help of the zip() function which results in a list of tuples. Here each item from list A is combined with corresponding elements from list B in the form of a tuple. Below is the syntax:

```
My_list = zip(list_1, list_2)
```

Example:

```
# combining lists with the help of zip() function

my_list_1 = [5, 2, 90, 24, 10]

my_list_2 = [6, 3, 91, 25, 12]

# combined

my_combined_list = list(zip(my_list_1, my_list_2))

print(my_combined_list)
```

Output:

```
[(5, 6), (2, 3), (90, 91), (24, 25), (10, 12)]
```

Finding the most common item

To find the most frequent element we make use of the set() function. The set() function removes all the duplicates from the list, and the max() function returns the most frequent element (which is found with the help of 'key'). The key is an optional single argument function. Below is the syntax:

```
Most_frequent_value = max(set(my_list), key=mylist.count)
```

Example :

```
# to find the most frequent element from the list

my_list = ['a', 'a', 'a', 'b', 'c', 'd', 'd', 'e']

most_frequent_value = max(set(my_list), key=my_list.count)

print("The most common element is:", most_frequent_value)
```

Output:

The most common element is: a

Checking for membership in a list

To check whether an item exists in a list, we use the *in* statement.

Example:

```
my_list = ['a', 'a', 'a', 'b', 'c', 'd', 'd', 'e']

# to check whether 'c' is a member of my_list

# returns true if present

print('c' in my_list)

# to check whether 'f' is a member of my_list

# returns false if not present

print('f' in my_list)
```

Output:

True

False

Flatten a list of lists

Sometimes we encounter a list where each element in itself is a list. To convert a list of lists into a single list, we use list comprehension.

```
my_list = [item for List in list_of_lists for item in List ]
```

Example:

```
# to flatten a list_of_lists by using list comprehension
```

```
list_of_lists = [[1, 2],
```

```
                 [3, 4],
```

```
                 [5, 6],
```

```
                 [7, 8]]
```

```
# using list comprehension
```

```
my_list = [item for List in list_of_lists for item in List]
```

```
print(my_list)
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

Python - List Comprehension

List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

The Syntax

```
newlist = [expression for item in iterable if condition == True]
```

The return value is a new list, leaving the old list unchanged.

Condition

The *condition* is like a filter that only accepts the items that evaluate to **True**.

Iterable

The *iterable* can be any iterable object, like a list, tuple, set etc.

Expression

The *expression* is the current item in the iteration, but it is also the outcome, which you can manipulate before it ends up.

Example:

Based on a list of fruits, you want a new list, containing only the fruits with the letter "a" in the name.

Without list comprehension you will have to write a **for** statement with a conditional test inside:

Example

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
newlist = []
```

```
for x in fruits:  
    if "a" in x:  
        newlist.append(x)
```

```
print(newlist)
```

With list comprehension you can do all that with only one line of code:

Example

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
```

```
newlist = [x for x in fruits if "a" in x]
```

```
print(newlist)
```

Output:

```
['apple', 'banana', 'mango']
```

Tuples in Python

Python Tuple is a collection of objects separated by commas. A tuple is similar to a Python list in terms of indexing, nested objects, and repetition but the main difference between both is Python tuple is immutable, unlike the Python list which is mutable.

What is Immutable in Tuples?

Unlike Python lists, tuples are immutable. Some Characteristics of Tuples in Python.

- Like Lists, tuples are ordered and we can access their elements using their index values
- We cannot update items to a tuple once it is created.
- Tuples cannot be appended or extended.
- We cannot remove items from a tuple once it is created.

Creating a Tuple in Python

Tuples in Python can only be created when they are being assigned, hence placing all the elements inside the parenthesis, separated by a comma will create a tuple. Let's take a closer look at the syntax:

```
tempTuple = ('apple', 'mango', 'banana')  
print(tempTuple)
```

Output

```
('apple', 'mango', 'banana')
```

The parenthesis in the above syntax are optional and hence a tuple can also be created without writing the parenthesis. But remember it's always a good practice to have the parenthesis, as it always increases the code readability and understanding.

The above snippet can also be written as:

```
tempTuple = 'apple', 'mango', 'banana'  
print(tempTuple)
```

Output

```
('apple', 'mango', 'banana')
```

Immutable in Tuples

Tuples in Python are similar to lists but with a key difference: they are immutable. This means once a tuple is created, its elements cannot be changed, added, or removed. Let's break down some characteristics of tuples:

- **Immutable Nature:** Once a tuple is created, its elements cannot be modified. This ensures data integrity and stability.
- **Ordered Collection:** Like lists, tuples maintain the order of elements as they are inserted.
- **Support for Duplicate Values:** Tuples allow duplicate values just like lists.
- **Accessing Elements:** Elements in a tuple can be accessed using their index values. Here's an example illustrating the immutability of tuples in python:

```
my_tuple = (1, 2, 3)

#Attempting to change an element in the tuple will result in an error
my_tuple[1] = 1 # This will raise a TypeError
```

When attempting to modify an element within a tuple, Python will raise a `TypeError` because tuples in python do not support item assignment due to their immutable nature. This constraint ensures that once data is stored in a tuple, it remains unchanged throughout the program's execution.

Python Tuple Types

Python offers two primary types of tuples: named tuples and unnamed tuples.

- **Named tuples:** created by subclassing the tuple class and assigning a name to the new class, serve well for representing structured data like records, akin to database entries. Each element within a named tuple corresponds to a field in the record, permitting access by name instead of index. For instance:

```
class MyTuple(tuple):
    pass

my_tuple = MyTuple(("one", "two", "three"))
print(my_tuple.one)
```

Outputs

one

- **Unnamed Tuples:** are more prevalent and are generated simply by separating values with commas. They are suitable for storing unstructured data that doesn't necessitate naming. Here's an example:

```
my_tuple = (1, 2, 3)
print(my_tuple)
```

Outputs

```
(1, 2, 3)
```

In this instance, a tuple in python containing three elements is created. Accessing elements within the tuple can be achieved using index notation, akin to how one would interact with a list.

Accessing Elements in a Python Tuple and Indexing

Accessing elements in a tuple is no different then accessing elements in a list. As python follows 0 based indexing hence a tuple with n elements will have indices from 0 through n-1. An index in a tuple is accessed with the index operator []. For example:

Let's consider a basic tuple:

```
tempTuple = ('hello', 1, 2, 3)
print(tempTuple[0]) # prints first element of the tuple
print(tempTuple[3]) # prints last element of the tuple
print(tempTuple[4]) # error
```

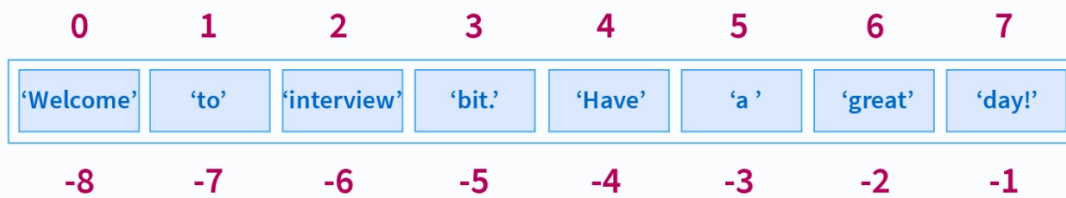
Nested Python Tuple Accessibility

```
nestedTuple = ('hello', [1, 2, 3], (4, 5, 6))
print(nestedTuple[0][2])
print(nestedTuple[2][2])
```

Accessing Via Negative Indices

Python allows you to access elements of a collection via negative indices. When accessing using a negative index, -1 depicts the last element and -n depicts the first index where n is the length of the index.

Consider the following mapping of positive index with negative index:



```
tempTuple = ('Welcome', 'to', 'interview', 'bit.', 'Have', 'a', 'great', 'day')
print(tempTuple[2]) # interview
print(tempTuple[-6]) # interview
```

Updating Tuples in Python

Adding a new element or deleting one is not really an option when dealing with tuples in python, as they are immutable. Even the elements of the tuple cannot be updated until and unless the element is mutable for example a list.

Let's take an example

```
tempTuple = ('Welcome', 'to', 'interview', 'bit.', 'Have', 'a', 'great', 'day', [1, 2, 3])
# tempTuple[0] = 'Hello' # throws type error, tuple object does not support type
# assignment.
tempTuple[8].append(4) # appending a new integer i.e. 4 in a list at 8th index of
# the tuple 'tempTuple'
# Printing the list at 8th index in the tuple
print(tempTuple[8]) # OUTPUT: [1, 2, 3, 4]
tempTuple[8].pop(3) # popping element at 3rd index from the list i.e. 8th index of
# the tuple 'tempTuple'
# Printing the list at 8th index in the tuple
print(tempTuple[8]) # OUTPUT: [1, 2, 3]
tempTuple = (1, 2, 3) # Assigning tuple all over again
# Printing the tuple
print(tempTuple) # OUTPUT: (1, 2, 3)
```

Output

```
[1,2,3,4]
[1,2,3]
```

```
(1,2,3)
```

Tuples in python can definitely be reassigned, which is very different from updating a tuple. Reassigning a tuple is redefining the tuple all over again.

Just like strings, we can also concat two or more tuples to form a new tuple using '+' operation or apply repetition on a tuple using '*' operator, just the result here is a python tuple and not a string.

```
# concatenating two different tuples
print(('apple', 'mango') + (1, 2, 3)) # OUTPUT: ('apple', 'mango', 1, 2, 3)
# repeat a tuple 5 times
print(("apple",) * 5) # OUTPUT: ('apple', 'apple', 'apple', 'apple', 'apple')
```

Output

```
('apple', 'mango', 1, 2, 3)
('apple', 'apple', 'apple', 'apple', 'apple')
```

In-built Functions for Tuple

Python has the following inbuilt functions to offer for tuples:

Function	Description
 cmp(tuple1,tuple2)	Compares elements of two different tuples.
 len(tuple)	Returns the length of the tuple
 max(tuple)	Returns the element with max value from the tuple.
 min(tuple)	Returns the element with min value from the tuple.
 tuple(seq)	Returns a tuple, by converting a list to a tuple. Takes a list in the parameter.

Different Operations Related to Tuples

- **Concatenation:** We will use plus operators(+) to Concatenation of Python Tuples.

```
tempTuple = ('apple', 'mango')
result = tempTuple + (1, 2, 3) # concatenating two different tuples
print('concatenation of a tuple', result)
# OUTPUT: concatenation of a tuple ('apple', 'mango', 1, 2, 3)
```

- **Nesting:** We can do the nesting of tuples means a tuple inside another tuple.``

```
# Code for creating nested tuples
tup1 = ('a','b','c')
tup2 = ('d','e')
tup3 = (tup1, tup2)
print(tup3)

#(('a','b','c'), ('d','e'))
```

- **Repetition:** We have the ability to form a tuple containing multiple identical elements by duplicating a single element within that tuple.

```
tup = ('2',)*3
print(tup)
#output: ('2','2','2')
```

- **Slicing:** Slicing in tuples works the same as it works for a [String slicing](#) or any other sequence of elements. Slice is an operator that allows you to fetch a sub collection (in this case a sub tuple) from a collection by slicing it from a start index and a stop index.

Slice syntax:

tuple[start : stop : step]

- **start:** is the starting index of the string, on which slicing operation has to be performed. It determines from where slicing of the string will 'begin'.
- **stop:** is the stopping index of the slicing, 'until' which slicing operation has to be performed i.e stop index is excluded while generating the sub-tuple.
- **step:** It is an optional argument that defines the steps when iterating the list i.e. it allows us to skip over elements.

Consider the above figure when understanding the following code snippet

```
temptuple = ("Welcome", "to", "interview", "bit.", "Have", "a", "great", "day")
tuple1 = temptuple[:] # fetching complete tuple
print("tuple1:", tuple1) # OUTPUT: ('Welcome', 'to', 'interview', 'bit.', 'Have', 'a', 'great', 'day')
tuple2 = temptuple[0 : 6] # fetching tuple from 0th index to 6th index
print("tuple2:", tuple2) # OUTPUT: ('Welcome', 'to', 'interview', 'bit.', 'Have', 'a')
tuple3 = temptuple[:: 3] # jumping every third element from start to end
print("tuple3:", tuple3) # OUTPUT: ('Welcome', 'bit.', 'great')
tuple4 = temptuple[1:5:2] # jumping to every 2nd element starting from 1st index until 5th index
print("tuple4:", tuple4) # OUTPUT: ('to', 'bit.')
tuple5 = temptuple[-8:-5] # 8th index from end to 5th index from end
print("tuple5:", tuple5) # OUTPUT: ('Welcome', 'to', 'interview')
tuple6 = temptuple[::-3] # jumping every 3rd element in reverse
print("tuple6:", tuple6) # OUTPUT: ('day', 'Have', 'to')
tuple7 = temptuple[-7:-3:2] # alternate implementation of tuple4
print("tuple7:", tuple7) # OUTPUT: ('to', 'bit.')
```

Output

```
tuple1: ('Welcome', 'to', 'interview', 'bit.', 'Have', 'a', 'great', 'day')
tuple2: ('Welcome', 'to', 'interview', 'bit.', 'Have', 'a')
tuple3: ('Welcome', 'bit.', 'great')
tuple4: ('to', 'bit.')
tuple5: ('Welcome', 'to', 'interview')
tuple6: ('day', 'Have', 'to')
tuple7: ('to', 'bit.')
```

- **Deleting:** As discussed, python tuples being immutable cannot be updated. Hence once some values are assigned to a tuple, it cannot be deleted. You can delete a tuple as a whole, but deleting a specific value/element in a tuple is not possible.

```
tempTuple = (1, 2, 3, 4, 5)
# tempTuple.pop() # throws error as object has no attribute pop
# del tempTuple[3] # throws error as tuple does not support object deletion
```

```
print(tempTuple) # OUTPUT: (1, 2, 3, 4, 5)
del tempTuple
print(tempTuple) # throws NameError: name 'tempTuple' is not defined
```

Output

```
(1,2,3,4,5)
Traceback (most recent call last):
  File "main.py", line 6, in <module>
    print(tempTuple) # throws NameError: name 'tempTuple' is not defined
NameError: name 'tempTuple' is not defined
```

- **Finding the length:** In Python, determining the length of a tuple can be accomplished using the built-in function `len()`. Simply provide the tuple as an argument to `len()`. This will return the number of elements contained within the tuple.

```
tup = (1,2)
print(len(tup))
#Output: 2
```

- **Multiple Data Types with tuples:** Tuples in Python are versatile containers, allowing you to store elements of different data types within the same tuple.

```
temp = ("a",True,2)
print(temp)
#Output: ('a',True,2)
```

- **Conversion of lists to tuples:** In Python, you can transform a list into a tuple effortlessly by utilizing the `tuple()` constructor and providing the list as its argument.

```
list1 = [1, 2, 3]
print(tuple(list1))
#Output: (1, 2, 3)
```

- **Tuples in a Loop:** We can also create a tuple with a single element in it using loops.

```
temp = ('a',)
n = 2
for i in range(int(n)):
    temp = (temp,)
    print(temp)
```

Output:

```
((('a',),),)
```

Converting a List to a Tuple

We can convert a [list](#) in Python to a tuple by using the tuple() constructor and passing the list as its parameters.

Code for converting a list and a string into a tuple

```
a = [0, 1, 2]
t = tuple(a)
print(t)
```

Output

```
(0, 1, 2)
```

Different Ways of Creating a Tuple

- Using round brackets
- Without Brackets
- Tuple Constructor
- Empty Tuple
- Single Element Tuple
- Using Tuple Packing

Using Round Brackets

```
t = ("gfg", "Python")
```

```
print(t)
```

Output

```
('gfg', 'Python')
```

Using Comma Separated

```
# Creating a tuple without brackets
```

```
t = 4, 5, 6
```

```
print(t) # Output: (4, 5, 6)
```

Output

```
(4, 5, 6)
```

Using Tuple Constructor

```
# Creating a tuple using the tuple() constructor
```

```
t = tuple([7, 8, 9])
```

```
print(t) # Output: (7, 8, 9)
```

Output

```
(7, 8, 9)
```

Creating an Empty Tuple

```
# Creating an empty tuple
```

```
t = ()
```



```
print(t) # Output: ()
```

Output

```
()
```

Single Element Tuple

Creating a single-element tuple

```
t = (10, ) # Comma is important here
```

```
print(t) # Output: (10,)
```

```
print(type(t))
```

What if we do not use comma

```
t = (10) # This an integer (not a tuple)
```

```
print(t)
```

```
print(type(t))
```

Output

```
(10,)
```

```
<class 'tuple'>
```

```
10
```

```
<class 'int'>
```

Tuple Packing

Tuple packing

```
a, b, c = 11, 12, 13
```

```
t = (a, b, c)
```

```
print(t) # Output: (11, 12, 13)
```

Output

```
(11, 12, 13)
```

Advantages and Disadvantages of Tuple in Python

Advantages

- Tuples being immutable, turns out to be a write-protected collection. Tuples can be of advantage when we want to store some secure read only data that we cannot afford to be changed throughout our code.
- Tuples can store data of multiple data types, that makes them a heterogeneous collection.
- Tuple being a readonly collection, has a faster iteration. (As they are stored in a single block of memory, and don't have extra space for storing objects, they have constant set of values)

Disadvantages

- Tuple's being write protected, is an advantage but also a disadvantage as it cannot be used when we want to add or delete a specific element. Hence has a limited use case.
- Syntactically less readable as, tuples can be created by either adding parentheses or by not providing them incase we have more than one element. But not using parentheses in case of one element, will not create a tuple and hence a trailing comma in such case is required. This can makes code readability a bit complex for some.
- As tuple is a class, it's stored on the heap and is overhead on the garbage collector.

Sets in Python

Sets in Python are a collection of unique elements that are unordered and immutable. In other words, sets can be thought of as a collection of distinct items that cannot be modified once created. Python provides a built-in set data type that allows users to create, manipulate and perform operations on sets efficiently. Sets in Python are very useful for various applications such as removing duplicates from a list, performing mathematical operations like union, intersection and difference on sets, etc.

Sets in python are also similar but with a few conditions.

- First, that a set cannot contain duplicate elements.
- Second, the elements of a set are immutable(once entered cannot be changed) but the set as a whole is mutable.

A Set in Python is used to store a collection of items with the following properties.

- No duplicate elements. If try to insert the same item again, it overwrites previous one.
- An unordered collection. When we access all items, they are accessed without any specific order and we cannot access items using indexes as we do in lists.
- Internally use hashing that makes set efficient for search, insert and delete operations. It gives a major advantage over a list for problems with these operations.
- Mutable, meaning we can add or remove elements after their creation, the individual elements within the set cannot be changed directly.

Creating Sets in Python

Just like most other things, creating a set in Python is really simple. There are two ways of creating a set

1. Putting all the elements inside curly braces “{“ and “}”
2. Using set() method.

If the curly braces look familiar to you, it's because they are used in Python Dictionaries.

Using Curly Braces

Let's look at an example.

```
evenNumber = {2, 4, 6}
print(evenNumber)
stringAndNum = {"one", 2, 3}
print(stringAndNum)
```

Output:

```
{2, 4, 6}
{'one', 2, 3}
```

Using set() function

Syntax: set([iterable])

Return type: Empty is no argument is given otherwise a set or frozenset object

set() takes an **iterable as an argument**. In python, string, lists, and dictionaries are iterable so you can pass them inside set(). set() returns a set or frozenset object (explained later in the article). If there is no input (i.e. no iterable passed as argument) then an empty set is returned.

Here is an example which uses set() function

```
setWithMethod = set([1, 2, 'three'])
print(setWithMethod)
```

Output:

```
{1, 2, 'three'}
```

Here is a code example that shows how you can make a set from iterables (lists, dictionaries, string, etc.)

```
a = {"one": 1, "two": 2}
b = [100, 200, 300]
c = 'python'
print(set(a), end='\n')
```

```
print(set(b), end='\n')
print(set(c))
```

Output:

```
{'one' , 'two'}
{200, 100, 300}
{'h', 'n', 'o', 'p', 'y', 't'}
```

One important property of set() is that the elements are immutable. So you cannot enter any mutable object inside a set (like dictionaries, lists, set).

```
# This wont work
mySetOne = {1, 2, [3, 4]}
print(mySetOne)
# This will work
mySetTwo = {1, 2, (3, 4)}
print(mySetTwo)
```

Here when mySetOne is executed it gives *TypeError: unhashable type: 'list'*

```
mySetOne = {1, 2, [3,4]}
TypeError: unhashable type: 'list'
```

When mySetTwo is executed, we get

```
{1, 2, (3,4)}
```

Check unique and Immutable with Python Set

Python sets cannot have duplicate values. While you cannot modify the individual elements directly, you can still add or remove elements from the set.

a set cannot have duplicate values

```
s = {"Good", "for", "Good"}
print(s)
```

values of a set cannot be changed

```
s[1] = "Hello"
print(s)
```

Output:

The first code explains that the set cannot have a duplicate value. Every item in it is a unique value.

The second code generates an error because we cannot assign or change a value once the set is created. We can only add or delete items in the set.

```
{'Good', 'for'}
```

```
TypeError: 'set' object does not support item assignment
```

Heterogeneous Element with Python Set

Python sets can store heterogeneous elements in it, i.e., a set can store a mixture of string, integer, boolean, etc datatypes.

```
# Python example demonstrate that a set  
# can store heterogeneous elements  
s = {"Good", "for", 10, 52.7, True}  
print(s)
```

Output

```
{True, 'for', 'Good', 10, 52.7}
```

Adding Items in a Python Set

Adding elements in a set is not the same as adding elements in lists. **Set do not use indexing.** There are two methods to add elements in a set

1. add() allows adding single element
2. update() allows to add multiple elements

Let's take a look at the code -

```
initialSet = {1, 2}  
initialSet.add(3)  
print(initialSet)  
toAdd = [4, 5]  
initialSet.update(toAdd)  
print(initialSet)
```

Output:

```
{1, 2, 3}  
{1, 2, 3, 4, 5}
```

update() takes an iterable like string, list, dictionary as an argument.

Removing Items from a Set in Python

For removing elements we have these methods:

1. **remove(element)** - This method removes the element from the set. If the element is not present then it will throw an error.
2. **discard(element)** - This method removes the element from the set. If the element is not present then the set will remain unchanged.
3. **clear()** - This method is used to delete all the set elements.

Let's take a look at the code to understand these method better.

```
mySet = {1, 2, 3, 5}
print("Before: ", mySet)
mySet.remove(3)
print("After: ", mySet)
mySet.discard(4)
print("Using discard: ", mySet)
mySet.clear()
print(mySet)
```

Output:

```
Before: {1, 2, 3, 5}
After: {1, 2, 5}
Using: {1, 2, 5}
set()
```

Here you can see that the element that is passed to `discard()` does not exist in the set and `discard()` did not give any error. Also, this code shows the use of `clear()`.

Set Operations in Python

Most operations on set can be performed in two ways.

1. Using **operator**
2. Using **methods**

We will look at both ways of accomplishing the task.

Union

Union operation combines all the elements from both sets. “|” (pipe) symbol is used to carry out this operation. We can also use the `union()` method in Python.



Lets look at the code example:

```
budgetCars = {'Maruti', 'Hyundai', 'Ford'}
highEndCard = {'Audi', 'BMW', 'Ford'}
print(budgetCars | highEndCard)
print(budgetCars.union(highEndCard))
```

Output:

```
{ 'Hyundai' , 'Ford' , 'Audi' , 'Maruti' , 'BMW' }
{ 'Hyundai' , 'Ford' , 'Audi' , 'Maruti' , 'BMW' }
```

In this example, you can see that union combines all the elements from both sets and it does not include any duplicate elements.

Intersection

Intersection operation picks the elements that are common in both sets. “&” operator or `intersection()` method is used to perform this operation.



Here in the example, you can see that the intersection result picks the two elements which are common in both the chemicals.

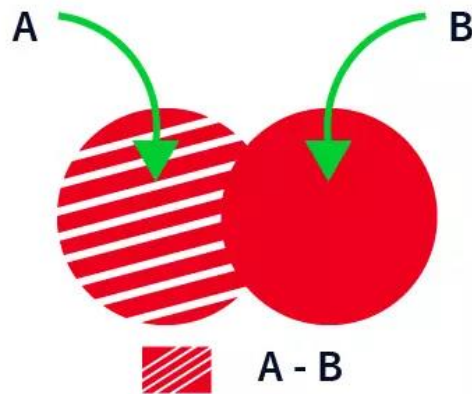
```
chemicalOne = {'Na', 'K', 'Cl'}  
chemicalTwo = {'HCL', 'Cl', 'Ba'}  
print(chemicalOne & chemicalTwo)  
print(chemicalOne.intersection(chemicalTwo))
```

Output:

```
{'Cl'}  
{'Cl'}
```

Difference

The difference in sets is similar to subtraction. When set B is subtracted from set A i.e, Set A - Set B then we say that in the resulting set all the elements of Set B will be removed from Set A. “-” (subtraction sign) or difference() method is used to perform this operation.



```
easyToLearn = {'Python', 'JavaScript', 'C++'}  
hardToLearn = {'Assembly', 'C++'}  
print(easyToLearn - hardToLearn)  
print(easyToLearn.difference(hardToLearn))  
print('Changing order we get: ')  
print(hardToLearn - easyToLearn)  
print(hardToLearn.difference(easyToLearn))
```

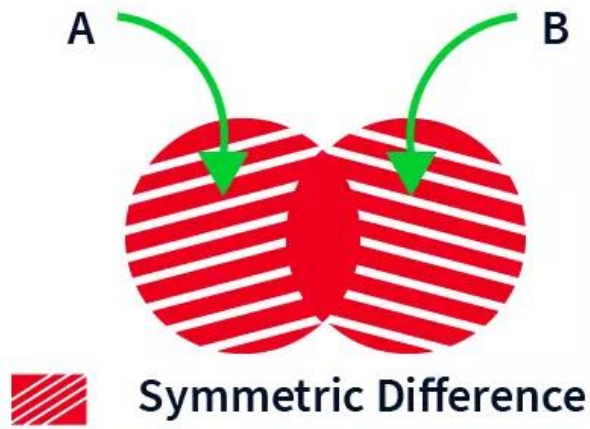
Output:

```
{'JavaScript' , 'Python'}  
{'JavaScript' , 'Python'}  
Changing order we get :  
{'Assembly'}  
{'Assembly'}
```

Let's assume that you passed three sets A, B, and C to find the difference. In this case, first, the difference of A and B will be calculated, then its result (suppose R) will be used to calculate the difference with C.

Symmetric Difference

The symmetric difference of two sets A and B is a set that contains elements in either A or B but not both. “^” operator or [symmetric_difference\(\) method in python](#) is used to perform this operation.



```
a = {1, 2, 3}
b = {3, 4, 5}
print(a ^ b)
print(a.symmetric_difference(b))
```

Output:

```
{1, 2, 4, 5}
{1, 2, 4, 5}
```

Frozen Set

Frozen set is a built-in data type in Python. It is a **set that is immutable** (once defined, cannot be changed). It supports all non-modifying operations of the set. Operations like `add()` won't work in the case of a frozen set.

Here is a simple example:

```
a = frozenset(['one', 'two', 'three'])
print(a)
```

Output:

```
frozenset({'one', 'two', 'three'})
```

A common use case of **frozenset** can be to store dictionary keys. Here is an example showing that

```
myDict = {'name': "Python", 'type': 'interpreted'}  
setOfDict = frozenset(myDict)  
print(setOfDict)
```

Output:

```
frozenset({'name', 'type'})
```

Suppose that we want to display this data to some user then using the **frozenset** makes sense as no one will be able to change them.

Python Built-in set methods

Method	Description
add()	Adds an item to the set if it is not already present.
clear()	Removes all the elements from the set
copy()	Returns a copy of the set.
difference()	Returns a set containing the difference between two or more sets
remove(item)	Remove the from the set; it must be a member. If the item is not a member, raise a KeyError.
discard()	Removes the specified item from the set.
difference_update()	Removes the items in this set that are also included in another, specified set.
intersection()	Returns a set, that is the intersection of two or more sets
intersection_update()	Removes the items in this set that are not present in other, specified set(s)
isdisjoint()	Returns whether two sets have a intersection or not
issubset()	Returns whether another set contains this set or not
issuperset()	Returns whether this set contains another set or not
pop()	Removes an element from the set
symmetric_difference()	Returns a set with the symmetric differences of two sets
symmetric_difference_update()	Inserts the symmetric differences from this set and another
union()	Return a set containing the union of sets

Method	Description
update()	Update the set with another set, or any other iterable

Dictionary in Python

A **Python dictionary** is a data structure that stores the value in **key: value** pairs. Values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and must be *immutable*.

The data is stored in a key-value pair format using Python Dictionary. In Python, a dictionary is a data type that may replicate a real-world data arrangement in which a given value exists for a specified key. It's a data structure that can be changed. The keys and values define a Dictionary.

Keys must be made up of only one element. Values can be of any type, including list, tuple, integer, etc. A dictionary, in other words, is a collection of key-value pairs with the value being any Python object. On the other hand, the keys are immutable Python objects such as Numbers, strings, or tuples.

Dictionary in Python is one of the most popular data structures. They are used to store data in a key-value pair format. The keys are always unique within a dictionary and are the attribute that helps us locate the data in the memory.

The values of the Python dictionary may or may not be unique. The values within a dictionary can be of any data type, but the thing to note is that the keys are immutable. Hence, the key can only be strings, numbers or tuples.

```
d = {1: 'Good', 2: 'For', 3: 'Good'}
print(d)
```

Output

```
{1: 'Good', 2: 'For', 3: 'Good'}
```

Two more things to keep in mind while working with a dictionary in Python –

- The keys must be only a single element.
- The keys are case-sensitive, i.e. the same name in either uppercase or lowercase will be treated differently.

Creating a Dictionary in Python

Creating a Python dictionary is as simple as placing the required key and value pairs within a curly bracket.

"{}".

A colon separates the key-value pair.

":"

When there are multiple key-value pairs, they are separated by a comma.

","

The syntax of declaring a dictionary in python is as follows –

```
my_dict={'Name':'Ravi',"Age":'32'}
```

Now let's look at some different ways of creating a Python dictionary –

```
#creating an empty dictionary
my_dict={}
#creating a dictionary with integer keys
fruits={'1':'apple','2':'banana','3':'cherry'}
#creating a dictionary with mixed keys
random_dict={'1':'red','Name':'Anushka'}
print(my_dict)
print(fruits)
print(random_dict)
```

Output –

```
{}
```

```
{'1': 'apple', '2': 'banana', '3': 'cherry'}
```

```
{'1': 'red', 'Name': 'Anushka'}
```

Another way we can create a dictionary in Python is using the built-in dict() function!

Let's see how we can do that –

```
Dict = dict([(1, 'Scaler'), (2, 'Apple')])
print("\nCreate a Dictionary by using dict(): ")
print(Dict)
```

Output –

```
Create a Dictionary by using dict():
{1: 'Scaler', 2: 'Apple'}
```

By using the dict() function, we could convert the value pairs in the round brackets into a dictionary.

Accessing values in Python Dictionary

We can use our favourite square brackets to access the values in a Python Dictionary! (Note- Square brackets are also commonly used for indexing and slicing strings in Python!)

Come along as we try to understand this better –

```
my_dict={'Name':'Ravi',"Age":'32','ID':'258RS569'}
print(my_dict['ID']) #accessing using the ID key
print(my_dict['Age']) #accessing using the Age
```

Output –

```
258RS569
32
```

In the above example, we could access the stored values using the ID and Age key, respectively.

If we tried to reference a key that is not present in our Python dictionary, we would get the following error –

```
print(my_dict['Phone']) #accessing using a key that is
not present
KeyError: 'Phone'
    print(my_dict['Phone'])
Line 4 in <module> (Solution.py)
```

Updating Dictionary in Python

We can do multiple things when trying to update a dictionary in Python. Some of them are –

1. Add a new entry.
2. Modify an existing entry.
3. Adding multiple values to a single key.
4. Adding a nested key.

Now let's try to implement these through code –

```
#creating an empty dictionary
my_dict={}
print(my_dict)
#adding elements to the dictionary one at a time
my_dict[1]="James"
my_dict[2]="Jim"
my_dict[3]="Jake"
print(my_dict)
#modifying existing entry
my_dict[2]="Apple"
print(my_dict)
#adding multiple values to a single key
my_dict[4]="Banana,Cherry,Kiwi"
print(my_dict)
#adding nested key values
my_dict[5]={'Nested' :{'1' : 'Scaler', '2' :
'Academy'}}
print(my_dict)
```

Output –

```
{}
```

```
{1: 'James', 2: 'Jim', 3: 'Jake'}
```

```
{1: 'James', 2: 'Apple', 3: 'Jake'}
```

```
{1: 'James', 2: 'Apple', 3: 'Jake', 4:
'Banana,Cherry,Kiwi'}
```

```
{1: 'James', 2: 'Apple', 3: 'Jake', 4:
'Banana,Cherry,Kiwi', 5: {'Nested': {'1': 'Scaler',
'2': 'Academy'}}}
```


Deleting Dictionary Elements

Now that we have covered how to update Python dictionaries, let's look at how we can either delete the dictionary entirely or remove individual entries.

To remove an entire dictionary in Python, we use the “del” keyword.

Let's check out its implementation –

```
my_dict={1: 'James', 2: 'Apple', 3: 'Jake', 4:
'Banana,Cherry,Kiwi'}
del my_dict[4]
print(my_dict)
```

Output –

```
{1: 'James', 2: 'Apple', 3: 'Jake'}
```

Using del

We deleted the entries for a specific key in the above example.

Now, if we want to clear an entire dictionary in Python, we will use the

[.clear\(\)](#)

Consider the following code –

```
my_dict={1: 'James', 2: 'Apple', 3: 'Jake', 4:
'Banana,Cherry,Kiwi'}
my_dict.clear()
print(my_dict)
```

Output –

```
{}
```

As you can see, the entire content of the dictionary “my_dict” was removed.

Removing Dictionary Items

We can remove items from dictionary using the following methods:

- `del`: Removes an item by key.
- `pop()`: Removes an item by key and returns its value.
- `clear()`: Empties the dictionary.
- `popitem()`: Removes and returns the last key-value pair.

```
d = {1: 'Good', 2: 'For', 3: 'Good', 'age':22}
```

```
# Using del to remove an item
```

```
del d["age"]
```

```
print(d)
```

```
# Using pop() to remove an item and return the value
```

```
val = d.pop(1)
```

```
print(val)
```

```
# Using popitem to removes and returns
```

```
# the last key-value pair.
```

```
key, val = d.popitem()
```

```
print(f"Key: {key}, Value: {val}")
```

```
# Clear all items from the dictionary
```

```
d.clear()
```

```
print(d)
```

Output

```
{1: 'Good', 2: 'For', 3: 'Good'}
```

```
Good
```

```
Key: 3, Value: Good
```

```
{}
```

Python Dictionary Comprehension

The concept of creating a new Python dictionary from an iterable is known as Python Dictionary Comprehension.



The diagram illustrates the components of a Python dictionary comprehension. The top line shows the general syntax: `{ key: value for vars in iterable }`. The bottom line shows a specific example: `{ num: num*num for num in range(1,11) }`. Vertical lines connect the parts of the example to the general syntax: 'num' connects to 'key', 'num*num' connects to 'value', 'for num' connects to 'for vars', and 'in range(1,11)' connects to 'in iterable'.

The syntax of python dictionary comprehension looks like this-

It consists of an expression pair (key: value) followed by a for statement in a curly brace.

Let's look at an example –

```
my_dict = {x: x*x for x in range(6)}  
print(my_dict)
```

Output –

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Using one code line, we created a dictionary within a range of 6, where the values are squares of the key!

The dictionary comprehension code would look something like this if we used a for loop –

```
my_dict = {}  
for x in range(6):  
    my_dict[x] = x*x  
print(my_dict)
```

We can also use an if statement with python dictionary comprehension to filter out values based on the conditions.

Iterating Dictionary in Python

A dictionary in Python can be easily iterated using a for loop.

We can do this in 4 ways –

1. Use the for loop to iterate the keys:

```
my_dict={1: 'James', 2: 'Apple', 3: 'Jake', 4:
'Banana,Cherry,Kiwi'}
for x in my_dict:
    print(x)
```

Output –

```
1
2
3
4
```

2. Use the for loop to iterate the values:

```
my_dict={1: 'James', 2: 'Apple', 3: 'Jake', 4:
'Banana,Cherry,Kiwi'}
for x in my_dict:
    print(my_dict[x])
```

Output –

```
James
Apple
Jake
Banana,Cherry,Kiwi
```

3. Use the for loop along with the items() method:

```
my_dict={1: 'James', 2: 'Apple', 3: 'Jake', 4:
'Banana,Cherry,Kiwi'}
for x in my_dict.items():
    print(x)
```

Output –

```
(1, 'James')
(2, 'Apple')
(3, 'Jake')
(4, 'Banana, Cherry, Kiwi')
```

4. Use the for loop along with the values() method:

```
my_dict={1: 'James', 2: 'Apple', 3: 'Jake', 4:
'Banana,Cherry,Kiwi'}
for x in my_dict.values():
    print(x)
```

Output –

```
James
Apple
Jake
Banana,Cherry,Kiwi
```

Properties of Python Dictionary Keys

I am sure by now you have realised that Python dictionary values have no restrictions and can be any Python object. The same is not true for keys.

So there are specific properties of keys we must be aware of to work with Python dictionaries efficiently –

1. Duplicate keys are not allowed. When duplicate keys are encountered in Python, the last assignment is the one that wins.

For example –

```
my_dict={1: 'James', 2: 'Apple', 3: 'Jake', 1:
'Banana,Cherry,Kiwi'}
print(my_dict)
```

Output –

```
{1: 'Banana, Cherry, Kiwi', 2: 'Apple', 3: 'Jake'}
```

2. Keys are immutable- they can only be numbers, strings or tuples.

Built-in Python Dictionary Functions & Methods

Dictionary Functions

Function	Description
<code>cmp(dict1,dict2)</code>	It compares the items of both the dictionaries and returns true if the value of the first dictionary is greater than the second else returns false.
<code>len(dict)</code>	It gives the total number of items in the dictionary.
<code>str(dict)</code>	It produces a printable string representation of the dictionary.
<code>all(dict)</code>	It returns true if all the keys in the dictionary are true.
<code>any(dict)</code>	It returns true if any key in the dictionary is true.
<code>sorted(dict)</code>	It returns a new sorted list of keys in a dictionary.

Python Dictionary Methods

Methods	Description
<code>dict.clear()</code>	It removes all elements of the dictionaries.
<code>dict.copy()</code>	It returns a copy of the dictionary.
<code>dict.pop()</code>	It removes an element from the specified key.
<code>dict.get()</code>	It is used to get the value of the specified key.
<code>dict.fromkeys()</code>	It creates a new dictionary with keys from seq and values set to value.
<code>dict.items()</code>	It returns the list of dictionary tuple pairs.
<code>dict.keys()</code>	It returns the list of keys of the dictionary.
<code>dict.update(dict2)</code>	It adds the dict2's key-value pairs to dict.
<code>dict.has_key()</code>	It returns true if the specified key is present in the dictionary else false.

Python Boolean Type

Boolean value can be of two types only i.e. either True or False. The output `<class 'bool'>` indicates the variable is a Boolean data type.

```
a = True
print(type(a))

b = False
print(type(b))
```

Output

```
<class 'bool'>
<class 'bool'>
```

Evaluate Variables and Expressions

We can evaluate values and variables using the **Python bool()** function. This method is used to return or convert a value to a Boolean value i.e., True or False, using the standard truth testing procedure.

Python bool() Function

In Python, the bool() function is used to convert a value or expression to its corresponding Boolean value (True or False). In the example below the variable res will store the boolean value of False after the equality comparison takes place.

Returns False as x is None

```
x = None
print(bool(x))
```

Returns False as x is an empty sequence

```
x = ()
print(bool(x))
```

Returns False as x is an empty mapping

```
x = {}
print(bool(x))
```

Returns False as x is 0

```
x = 0.0
print(bool(x))
```

```
# Returns True as x is a non empty string  
x = 'Good'  
print(bool(x))
```

Output

```
False  
False  
False  
False  
True
```

Note: It's not always necessary to use bool() directly. Python automatically evaluates expressions in terms of their Boolean values, especially when used in conditions like if statements or loops.

Integers and Floats as Boolean

In Python, integers and floats can be used as Boolean values with the bool() function. Any number with a value of zero (0, 0.0) is considered False while any non-zero number (positive or negative) is considered True.

```
var1 = 0  
print(bool(var1))  
  
var2 = 1  
print(bool(var2))  
  
var3 = -9.7  
print(bool(var3))
```

Output

```
False  
True  
True
```

Boolean Operators

Boolean Operations in Python are simple arithmetic of True and False values. These values can be manipulated by the use of boolean operators which include **AND or and NOT**. Common boolean operations are –

- or
- and
- not
- == (equivalent)
- != (not equivalent)

Boolean OR Operator

Boolean or operator returns True if any one of the inputs is True else returns False.

```
a = 5
```

```
b = 3
```

```
c = 8
```

```
if a > b or b < c:
    print("True")
```

Output

```
True
```

Explanation:

- The condition $a > b$ or $b < c$ uses the or operator, which returns True if any one of the conditions is True.
- $a > b$ is True because $5 > 3$ and $b < c$ is also True because $3 < 8$.
- Since at least one condition is True, the if block executes and “True” is printed.

Boolean And Operator

Boolean and operator returns False if any one of the inputs is False else returns True.

Example:

```
a = 0
```

```
b = 2
```

```
c = 4
```

```
if a > b and b < c:
    print(True)
else:
    print(False)
```

```
if a and b and c:
    print("True")
else:
    print("False")
```

Output

False

False

Explanation:

- In the first part, the condition $a > b$ and $b < c$ evaluates to False because $a > b$ is False, causing the else block to print False.
- In the second part, a is 0 (which is considered False), so the entire condition evaluates to False and the else block prints "False".

Boolean Not Operator

Boolean Not operator only requires one argument and returns the negation of the argument i.e. returns the True for False and False for True.

Example:

```
a = 0
```

```
if not a:  
    print("False")
```

Output

False

Explanation:

- The not operator inverts the Boolean value of the expression.
- Since $a = 0$ (which is considered False), not a evaluates to True.
- As a result, the condition if not a is true and the program prints "Boolean value of a is False".

Python Boolean == (equivalent) and != (not equivalent) Operator

Both operators are used to compare two results. '==' equivalent operator returns True if two results are equal and '!=' not equivalent operator returns True if the two results are not same.

Example:

```
a = 0
```

```
b = 1
```

```
if a == 0:  
    print(True)
```

```
if a == b:
```

```
print(True)
```

```
if a != b:  
    print(True)
```

Output

```
True
```

```
True
```

Explanation:

- The first condition `a == 0` is True because `a` is indeed 0, so it prints True.
- The second condition `a == b` is False because `a` (which is 0) is not equal to `b` (which is 1), so nothing is printed.
- The third condition `a != b` is True because `a` (which is 0) is not equal to `b` (which is 1), so it prints True.

Python is Operator

is keyword is used to test whether two variables belong to the same object. The test will return True if the two objects are the same else it will return False even if the two objects are 100% equal.

Example:

```
x = 10
```

```
y = 10
```

```
if x is y:  
    print(True)  
else:  
    print(False)
```

Output

```
True
```

Explanation:

- `is` keyword checks if two variables point to the same object in memory.
- In this case, both `x` and `y` are assigned the value 10. Python often optimizes memory for small integers, so both `x` and `y` reference the same object in memory.
- Since `x is y` evaluates to True, the if block is executed and True is printed.

Python in Operator

in operator checks for the membership i.e. checks if the value is present in a list, tuple, range, string, etc.

```
# Create a list
```

```
a = [1, 2, 2]
```

```
# Check if 1 is in list or not
```

```
if 1 in a:
```

```
    print(True)
```

Output

```
True
```

Explanation:

- in keyword checks if an element exists in a list.
- In this case, the element 1 is present in the list a = [1, 2, 2].
- Since 1 is found in the list, the if block executes and True is printed.